

# ADJSCC 潜在分布を補正して Stable Diffusion へ handover する Latent Resfusion

システムモデル・学習方法・評価計画

2026 年 8 月 17 日

## 概要

本メモは，Stable Diffusion の scaled VAE latent を ADJSCC で伝送したとき，復元潜在が VAE 潜在分布から外れる問題に対し，Resfusion を潜在空間へ導入するシステムを定式化する．想定する経路は

VAE encoder → ADJSCC encoder → wireless channel → ADJSCC decoder  
→ Latent Resfusion → Stable Diffusion → VAE decoder

である．Resfusion 原論文 [1] は RGB 画像復元を対象としており，潜在空間での利用と Stable Diffusion への途中 handover は本プロジェクト固有の拡張である．特に，Resfusion の途中状態をそのまま Stable Diffusion へ渡すと，残差項が残るため Stable Diffusion の forward training distribution には一致しない．そこで，残差推定 head と明示的な bridge を追加し，handover 状態を Stable Diffusion が学習した分布へ写す方法を提案する．

## 目次

|     |                                  |   |
|-----|----------------------------------|---|
| 1   | 目的と結論                            | 3 |
| 1.1 | 検証したい仮説                          | 3 |
| 1.2 | 本メモの推奨構成                         | 3 |
| 2   | 既存の分布測定が示す問題                     | 3 |
| 2.1 | 1000 画像での測定                      | 3 |
| 2.2 | forward diffusion で差が見えなくなることの解釈 | 4 |
| 3   | 記号とシステムモデル                       | 4 |
| 3.1 | 記号                               | 4 |
| 3.2 | end-to-end の信号経路                 | 5 |
| 3.3 | ADJSCC と wireless channel        | 5 |
| 4   | Resfusion の原式を latent へ移す        | 5 |
| 4.1 | forward process                  | 5 |
| 4.2 | Resfusion を劣化 latent から開始できる理由   | 6 |
| 4.3 | residual noise prediction        | 6 |

|      |                                                     |    |
|------|-----------------------------------------------------|----|
| 5    | <b>Stable Diffusion への途中 handover</b>               | 6  |
| 5.1  | 直接 handover が正しくない理由 . . . . .                      | 6  |
| 5.2  | 推奨 bridge：残差を明示的に除去する . . . . .                     | 7  |
| 5.3  | handover sample alignment loss . . . . .            | 7  |
| 5.4  | Stable Diffusion U-Net による interface loss . . . . . | 7  |
| 5.5  | moment/covariance loss は補助項とする . . . . .            | 8  |
| 5.6  | 総 loss . . . . .                                    | 8  |
| 5.7  | 別 schedule を使う場合の安全な bridge . . . . .               | 8  |
| 6    | <b>Latent Resfusion の network 設計</b>                | 9  |
| 6.1  | 入出力 . . . . .                                       | 9  |
| 6.2  | handover 範囲の学習 . . . . .                            | 9  |
| 7    | <b>学習手順</b>                                         | 9  |
| 7.1  | dataset pair の生成 . . . . .                          | 9  |
| 7.2  | 段階的 training . . . . .                              | 10 |
| 7.3  | 推奨する batch 内 sampling . . . . .                     | 10 |
| 8    | <b>推論手順</b>                                         | 11 |
| 8.1  | 共有 schedule・明示 bridge 方式 . . . . .                  | 11 |
| 8.2  | 疑似コード . . . . .                                     | 11 |
| 9    | <b>評価計画</b>                                         | 12 |
| 9.1  | 比較系 . . . . .                                       | 12 |
| 9.2  | 測定量 . . . . .                                       | 12 |
| 9.3  | 主要 ablation . . . . .                               | 12 |
| 10   | <b>成功条件と失敗の切り分け</b>                                 | 13 |
| 10.1 | 段階ごとの成功条件 . . . . .                                 | 13 |
| 10.2 | 想定される失敗と診断 . . . . .                                | 13 |
| 11   | <b>実装へ落とす際の変更点</b>                                  | 13 |
| 12   | <b>研究上の注意</b>                                       | 14 |
| 13   | <b>最小実験の実行順</b>                                     | 14 |

# 1 目的と結論

## 1.1 検証したい仮説

入力画像を  $x$ , Stable Diffusion の scaled VAE latent を  $z_0$ , ADJSCC 復元潜在を  $\hat{z}_0$  とする. 現在の仮説は次の連鎖である.

1.  $\hat{z}_0$  は  $z_0$  の分布, とくに分散とチャネル間共分散から外れる.
2. 通常の forward diffusion を適用しても, 中間時刻では  $\hat{z}_t$  と  $z_t$  の分布差が残る.
3. Stable Diffusion の U-Net は  $z_t$  上で学習されているため, out-of-distribution な  $\hat{z}_t$  では noise/score prediction 誤差が増える.
4. その誤差が逆拡散中に蓄積し, 顔・細部・色・質感の artifact となる.

## 1.2 本メモの推奨構成

最初の実装では, 以下を採用する.

- VAE, 学習済み ADJSCC, Stable Diffusion は凍結する.
- Resfusion を RGB ではなく  $4 \times 32 \times 32$  の scaled latent 上で学習する.
- Resfusion と Stable Diffusion で同じ 1000-step の raw timestep と  $\bar{\alpha}_t$  を共有する.
- Latent Resfusion は resnoise に加えて通信残差  $R = \hat{z}_0 - z_0$  を推定する.
- handover 時に  $z_t^H = v_t - (1 - \sqrt{\bar{\alpha}_t})\hat{R}_\theta$  という明示的 bridge を通す.
- 凍結した Stable Diffusion U-Net を教師として, handover 前後の noise prediction を一致させる loss を加える.

ここで  $v_t$  は Resfusion 側の状態であり, Stable Diffusion 側の通常状態  $z_t$  と区別する.

# 2 既存の分布測定が示す問題

## 2.1 1000 画像での測定

ADJSCC/outputs/latent\_distribution/C32\_best.json は, FFHQ validation 1000 画像, scaled VAE latent  $4 \times 32 \times 32$ , ADJSCC checkpoint sd\_vae\_ffhq\_stage1\_C32/best.pt を用いた測定である. 表 1 に主要量をまとめる. 共分散誤差は

$$e_{\text{cov}} = \frac{\|C_{\hat{z}} - C_z\|_F}{\|C_z\|_F}, \quad r_{\text{eff}}(C) = \frac{\text{tr}(C)^2}{\text{tr}(C^2)}$$

で定義した. 後者は 4 チャネルの分散が何方向へ実効的に広がっているかを表す participation ratio である.

高 SNR の 20 dB でも global std は clean の約 88.2%, 共分散 trace は約 76.3% であり, 単なる無線雑音だけでなく ADJSCC encoder-decoder 自体の縮退・バイアスが残っている. また  $r_{\text{eff}}$  の低下は, 4 チャネルの情報が一部の共分散固有方向へ偏っていることを示す. したがって, global mean/std だけの affine 補正では不十分である.

表 1 clean  $z_0$  と ADJSCC 復元  $\hat{z}_0$  の分布統計

| 条件          | global mean | global std | std 比 | $\text{tr}(C)/\text{tr}(C_z)$ | $e_{\text{cov}}$ | $r_{\text{eff}}$ |
|-------------|-------------|------------|-------|-------------------------------|------------------|------------------|
| clean $z_0$ | 0.0088      | 0.8546     | 1.000 | 1.000                         | 0.000            | 2.629            |
| −10 dB      | 0.0190      | 0.6414     | 0.750 | 0.534                         | 0.403            | 1.653            |
| −5 dB       | 0.0159      | 0.6917     | 0.809 | 0.635                         | 0.319            | 1.767            |
| 0 dB        | 0.0188      | 0.7265     | 0.850 | 0.707                         | 0.260            | 1.886            |
| 5 dB        | 0.0198      | 0.7424     | 0.869 | 0.740                         | 0.232            | 1.947            |
| 10 dB       | 0.0208      | 0.7499     | 0.877 | 0.756                         | 0.222            | 1.959            |
| 20 dB       | 0.0206      | 0.7534     | 0.882 | 0.763                         | 0.216            | 1.972            |

## 2.2 forward diffusion で差が見えなくなることの解釈

20 dB における clean  $z_t$  と ADJSCC  $\hat{z}_t$  の共分散相対誤差は,  $t = 0, 100, 250, 500, 750, 999$  でそれぞれ

$$0.215, \quad 0.193, \quad 0.144, \quad 0.0545, \quad 0.0106, \quad 0.0008$$

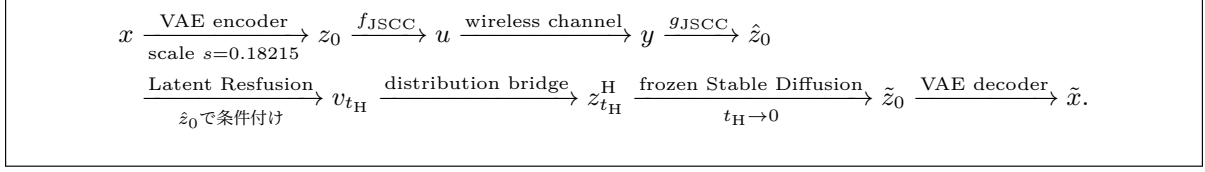
である. 大きな  $t$  で差が小さくなるのは, 両者が同じ標準 Gaussian noise に支配されるためである. これは意味内容が回復したことを意味しない. むしろ, handover を遅い時刻 (強い noise) に置けば distribution alignment は容易だが, 受信 latent に含まれる identity や細部を捨てやすいという trade-off を示す.

## 3 記号とシステムモデル

### 3.1 記号

| 記号                                             | 意味                                                        |
|------------------------------------------------|-----------------------------------------------------------|
| $x$                                            | 送信元画像                                                     |
| $z_0 \in \mathbb{R}^{4 \times H/8 \times W/8}$ | Stable Diffusion 用の scaled clean VAE latent               |
| $\hat{z}_0$                                    | ADJSCC decoder が復元した scaled latent                        |
| $R = \hat{z}_0 - z_0$                          | 通信系全体が生じさせた latent residual                               |
| $v_t$                                          | Resfusion の forward/reverse state                         |
| $z_t$                                          | Stable Diffusion 標準 forward process の state               |
| $t_\star$                                      | $\sqrt{\alpha_t} \simeq 1/2$ となる Resfusion 開始点            |
| $t_H$                                          | Resfusion から Stable Diffusion への handover raw timestep    |
| $c$                                            | Stable Diffusion の text condition (初期実験では null condition) |
| $\gamma$                                       | 実際の channel SNR (dB 表記の場合は $\gamma_{\text{dB}}$ )         |

### 3.2 end-to-end の信号経路



VAE encoder の posterior を sample する場合は

$$z^{\text{raw}} \sim q_{\text{VAE}}(z | x), \quad z_0 = s z^{\text{raw}}, \quad s = 0.18215$$

とし, clean 教師と ADJSCC 入力で必ず同一 sample を使う. posterior mean を使う実験では両方とも mean に統一する. scale の掛け忘れや二重適用は, 本研究で検出したい分布差と区別できなくなるため禁止する.

### 3.3 ADJSCC と wireless channel

ADJSCC encoder と decoder を

$$u = f_{\text{JSCC}}(z_0; \phi_e), \quad \hat{z}_0 = g_{\text{JSCC}}(y, \gamma; \phi_d)$$

とする. 複素 AWGN channel の例では

$$y = u + n_{\text{ch}}, \quad n_{\text{ch}} \sim \mathcal{CN}(0, N_0 I), \quad \frac{1}{n} \mathbb{E} \|u\|_2^2 \leq P, \quad \text{SNR} = \frac{P}{N_0}.$$

実装が real tensor で I/Q を 2 チャンネルとして表す場合も, noise variance の定義をこの式と整合させる. 学習時は SNR を固定せず, 対象範囲  $\gamma_{\text{dB}} \in [-10, 20]$  から sample し, 同じ画像を再訪するたびに新しい channel noise を生成する.

## 4 Resfusion の原式を latent へ移す

### 4.1 forward process

Resfusion 原論文 [1] の clean image を  $z_0$ , degraded image を  $\hat{z}_0$  に置き換える.

$$R = \hat{z}_0 - z_0, \quad \alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{i=0}^t \alpha_i.$$

ここから raw timestep は実装と同じ 0-origin  $t \in \{0, \dots, T-1\}$  で表す. 1-step 式の  $t=0$  は  $v_{-1} = z_0$  とみなす code-compatible な表記である. Resfusion の 1-step transition は

$$q(v_t | v_{t-1}, R) = \mathcal{N}(\sqrt{\alpha_t} v_{t-1} (1 - \sqrt{\alpha_t}) R, (1 - \alpha_t) I). \quad (1)$$

閉形式は

$$v_t = \sqrt{\bar{\alpha}_t} z_0 (1 - \sqrt{\bar{\alpha}_t}) R \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (2)$$

## 4.2 Resfusion を劣化 latent から開始できる理由

$R = \hat{z}_0 - z_0$  を式 (2) へ代入すると

$$v_t = (2\sqrt{\bar{\alpha}_t} - 1)z_0(1 - \sqrt{\bar{\alpha}_t})\hat{z}_0\sqrt{1 - \bar{\alpha}_t}\epsilon. \quad (3)$$

したがって

$$t_\star = \arg \min_t \left| \sqrt{\bar{\alpha}_t} - \frac{1}{2} \right|$$

では clean  $z_0$  の係数がほぼ 0 になり、未知の clean latent なしで

$$v_{t_\star} \simeq \sqrt{\bar{\alpha}_{t_\star}} \hat{z}_0 + \sqrt{1 - \bar{\alpha}_{t_\star}} \epsilon \quad (4)$$

を生成できる。これが Resfusion の加速点である。

本リポジトリの Stable Diffusion v1 設定は 1000 raw timesteps,  $\beta_{\text{start}} = 0.00085$ ,  $\beta_{\text{end}} = 0.012$  である。ldm/modules/diffusionmodules/util.py の make\_beta\_schedule("linear") と同じ計算をすると、0-origin index で

$$t_\star = 520, \quad \sqrt{\bar{\alpha}_{520}} = 0.50031$$

となる。なお、この「linear」schedule は端点の平方根を線形補間してから二乗する実装であり、 $\beta$  自体の単純な線形補間とは異なる。

## 4.3 residual noise prediction

Resfusion が予測する教師信号 (resnoise) を

$$\eta_t = \epsilon + \frac{(1 - \sqrt{\bar{\alpha}_t})\sqrt{1 - \bar{\alpha}_t}}{\beta_t} R \quad (5)$$

とする。原論文の基本 loss は

$$\mathcal{L}_{\text{RN}} = \mathbb{E}_{z_0, \hat{z}_0, t, \epsilon} [\|\eta_\theta(v_t, \hat{z}_0, t, \gamma) - \eta_t\|_2^2]. \quad (6)$$

逆過程の DDPM 形式 [2] は

$$v_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( v_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \eta_\theta(v_t, \hat{z}_0, t, \gamma) \right) + \sigma_t \xi, \quad (7)$$

$$\xi \sim \mathcal{N}(0, I), \quad \sigma_t^2 = \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t,$$

であり、最終 step では  $\xi = 0$  とする。

## 5 Stable Diffusion への途中 handover

### 5.1 直接 handover が正しくない理由

Stable Diffusion の U-Net が学習時に見る標準 state を

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \quad (8)$$

とする。Resfusion state との差は、式 (2) と (8) から

$$v_t - z_t = (1 - \sqrt{\bar{\alpha}_t})R \quad (9)$$

である。よって、Resfusion を何 step か実行したという理由だけで  $v_{t_H}$  をそのまま Stable Diffusion U-Net へ入力すると、通信残差に比例する distribution shift が残る。これは本研究が解消したい問題を handover 地点に持ち込むことになる。

## 5.2 推奨 bridge：残差を明示的に除去する

Latent Resfusion U-Net に resnoise head だけでなく residual head を設け、

$$\hat{R}_\theta = R_\theta(v_t, \hat{z}_0, t, \gamma)$$

を出力させる。handover state を

$$z_{t_H}^H = v_{t_H} - (1 - \sqrt{\bar{\alpha}_{t_H}})\hat{R}_\theta \quad (10)$$

と定義する。 $\hat{R}_\theta = R$  なら

$$z_{t_H}^H = \sqrt{\bar{\alpha}_{t_H}}z_0 + \sqrt{1 - \bar{\alpha}_{t_H}}\epsilon = z_{t_H},$$

となり、sample 単位でも Stable Diffusion の forward training state に一致する。平均・分散だけを合わせる補正より強い条件である。

残差 head の教師あり loss は

$$\mathcal{L}_R = \mathbb{E}\|\hat{R}_\theta - R\|_1 \quad \text{または} \quad \mathbb{E}\|\hat{R}_\theta - R\|_2^2. \quad (11)$$

外れ値の大きい latent では、初期実験に smooth  $L_1$  を用い、MSE との ablation を取る。

## 5.3 handover sample alignment loss

学習時には式 (2) と同じ Gaussian noise  $\epsilon$  で Stable Diffusion 教師 state

$$z_t^{\text{teacher}} = \sqrt{\bar{\alpha}_t}z_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

を作る。そして

$$\mathcal{L}_H = \mathbb{E}\left[w_t \left\|z_t^H - z_t^{\text{teacher}}\right\|_2^2\right] \quad (12)$$

を最小化する。同一 noise を使わないと、正しい bridge でも無関係な Gaussian 差を罰することになる。 $w_t$  は時刻間の scale 差を抑える重みであり、最初は  $w_t = 1$  としてよい。

## 5.4 Stable Diffusion U-Net による interface loss

凍結した Stable Diffusion U-Net を  $\epsilon_\psi^{\text{SD}}(z, t, c)$  とする。handover state が U-Net にとって同等であることを直接学習するため、

$$\mathcal{L}_{\text{score}} = \mathbb{E}\left[\left\|\epsilon_\psi^{\text{SD}}(z_t^H, t, c) - \text{sg}\{\epsilon_\psi^{\text{SD}}(z_t^{\text{teacher}}, t, c)\}\right\|_2^2\right]. \quad (13)$$

Stable Diffusion の parameter は `requires_grad=False` にするが、左側の student forward を `torch.no_grad()` で囲んではならない。U-Net の重みは更新しなくても、入力  $z_t^H$  から Resfusion へ gradient を戻す必要がある。teacher 側だけは no-grad または detach してよい。

初期実験では text condition  $c$  を empty prompt に固定し、classifier-free guidance scale は 1 とする。強い guidance は受信した identity より prompt prior を優先する可能性があり、distribution 補正そのものの効果を評価しにくくするためである。

## 5.5 moment/covariance loss は補助項とする

mini-batch と空間位置をまとめて 4 次元観測とみなし、 $\mu(\cdot) \in \mathbb{R}^4$ ,  $C(\cdot) \in \mathbb{R}^{4 \times 4}$  を計算する。

$$\begin{aligned} \mathcal{L}_{\text{stat}} = & \|\mu(z_t^{\text{H}}) - \mu(z_t^{\text{teacher}})\|_2^2 \\ & + \|\text{diag } C(z_t^{\text{H}}) - \text{diag } C(z_t^{\text{teacher}})\|_2^2 \\ & + \|C(z_t^{\text{H}}) - C(z_t^{\text{teacher}})\|_{\text{F}}^2. \end{aligned} \quad (14)$$

統計だけを一致させても画像ごとの意味対応は保証されないため、 $\mathcal{L}_{\text{stat}}$  は  $\mathcal{L}_R, \mathcal{L}_H, \mathcal{L}_{\text{score}}$  の代替ではなく小さい重みの補助項とする。小 batch では共分散推定が不安定なので、空間位置を観測として使うか、EMA covariance を用いる。

## 5.6 総 loss

推奨する総 loss は

$$\mathcal{L} = \lambda_{\text{RN}} \mathcal{L}_{\text{RN}} + \lambda_R \mathcal{L}_R + \lambda_H \mathcal{L}_H + \lambda_{\text{score}} \mathcal{L}_{\text{score}} + \lambda_{\text{stat}} \mathcal{L}_{\text{stat}} + \lambda_{\text{img}} \mathcal{L}_{\text{img}}. \quad (15)$$

$\mathcal{L}_{\text{img}}$  は任意であり、凍結 VAE decoder を介した  $L_1 + \text{LPIPS}$  等とする。VAE decoder も重みは凍結するが、この loss を用いる場合は入力 latent への gradient を切らない。

段階導入時の初期値として

$$\lambda_{\text{RN}} = 1, \quad \lambda_R = 1, \quad \lambda_H = 1, \quad \lambda_{\text{score}} = 0$$

から始め、latent 復元が安定した後に  $\lambda_{\text{score}}$  を徐々に上げる。 $\lambda_{\text{stat}}$  と  $\lambda_{\text{img}}$  は 0 から ablation で追加する。各 loss の絶対 scale が異なるため、数値は固定の結論ではなく validation で調整する。

## 5.7 別 schedule を使う場合の安全な bridge

公式 Resfusion 実装 [5] のように短い独自 schedule (例:  $T = 12$ , 実質約 5 復元 step) を使う場合、その時刻 index を Stable Diffusion の raw timestep として渡してはならない。Resfusion から clean latent 推定  $\tilde{z}_{0,\theta}$  を得て、Stable Diffusion の handover raw timestep  $\tau$  へ

$$z_\tau^{\text{H}} = \sqrt{\bar{\alpha}_\tau^{\text{SD}}} \tilde{z}_{0,\theta} + \sqrt{1 - \bar{\alpha}_\tau^{\text{SD}}} \epsilon_{\text{H}}, \quad \epsilon_{\text{H}} \sim \mathcal{N}(0, I) \quad (16)$$

と明示的に re-noise する。この方式は時計の混同を防げる一方、新しい noise により受信細部を失いやすい。まず式 (10) の共有 schedule 方式を正しさの baseline とし、速度が必要になった段階で式 (16) を比較する。



## 6 Latent Resfusion の network 設計

### 6.1 入出力

公式 RDDM U-Net は RGB の 3 チャンネルを前提にしている．本システムでは

$$v_t \in \mathbb{R}^{B \times 4 \times 32 \times 32}, \quad \hat{z}_0 \in \mathbb{R}^{B \times 4 \times 32 \times 32}$$

を channel 方向に結合するため，最初の convolution は 8 入力チャンネルとする．共有 backbone から

$$\eta_\theta \in \mathbb{R}^{B \times 4 \times 32 \times 32}, \quad \hat{R}_\theta \in \mathbb{R}^{B \times 4 \times 32 \times 32}$$

を出す two-head 構成，または 8 出力チャンネルを 4+4 へ分割する構成にする．SNR  $\gamma$  は sinusoidal/MLP embedding 後に各 ResBlock へ FiLM で注入する．

重要な差分は次のとおりである．

- RGB 用の  $[0, 1] \rightarrow [-1, 1]$  変換を latent へ適用しない．
- scaled VAE latent は非有界なので  $[-1, 1]$  clamp をしない．
- official RGB checkpoint の first/last convolution だけを変更してそのまま転用することを前提にしない．domain が異なるため，まず latent pair で学習する．
- time embedding には DDIM の step 番号ではなく Stable Diffusion の raw timestep  $t \in [0, 999]$  を渡す．

### 6.2 handover 範囲の学習

Resfusion 開始点を  $t_\star = 520$  とし，handover 候補を

$$t_H \in \{500, 250, 100\}$$

から検証する．学習では

$$t \sim \mathcal{U}\{t_H, \dots, t_\star\}$$

を基本とし，handover loss は候補時刻で重点的に計算する． $t_H = 500$  は Stable Diffusion へ早く渡すが ADJSCC 情報の補正量が小さく， $t_H = 100$  は Resfusion が長く復元を担当する．最適時刻は SNR 依存の可能性があるので，将来は

$$t_H = h(\gamma, \text{uncertainty})$$

とする adaptive handover も検討する．ただし，最初は固定時刻で因果を切り分ける．

## 7 学習手順

### 7.1 dataset pair の生成

各 mini-batch で次を行う．

1. 画像  $x$  を凍結 VAE encoder へ通し，scaled latent  $z_0$  を一度だけ作る．

2. SNR  $\gamma$  を sample し, 凍結 ADJSCC encoder, fresh channel noise, 凍結 ADJSCC decoder を通して  $\hat{z}_0$  を作る.
3.  $R = \hat{z}_0 - z_0$  を教師として保持する.
4. raw timestep  $t$  と  $\epsilon \sim \mathcal{N}(0, I)$  を sample する.
5. 同一の  $\epsilon$  で Resfusion state  $v_t$  と Stable Diffusion 教師 state  $z_t^{\text{teacher}}$  を作る.
6. network から  $\eta_\theta, \hat{R}_\theta$  を得て, 式 (15) を計算する.

pair を事前保存する方法より online channel simulation を優先する. これにより同一画像に対して SNR/noise realization を増やせる. 一方, validation では画像 split, SNR, channel noise seed, diffusion noise seed を固定し, 全モデルを同一条件で比較する.

## 7.2 段階的 training

■Phase 0 : 式と scale の unit test network 学習前に, 真の  $R$  を式 (10) へ入れたとき  $z_t^{\text{H}} = z_t^{\text{teacher}}$  が数値誤差内で成立することを確認する. さらに  $t_* = 520$ , shape, dtype, VAE scale, raw timestep を test する.

■Phase 1 : Resfusion 本体 VAE, ADJSCC, Stable Diffusion を凍結し,  $\mathcal{L}_{\text{RN}} + \mathcal{L}_R + \mathcal{L}_H$  で Latent Resfusion だけを学習する. optimizer は原論文に合わせて AdamW を出発点とし, learning rate  $1.1 \times 10^{-4}$  を候補にするが, latent U-Net の batch size に合わせて validation で調整する.

■Phase 2 : Stable Diffusion interface alignment Phase 1 checkpoint から開始し,  $\mathcal{L}_{\text{score}}$  を追加する. 最初は handover 近傍の raw timestep だけで計算し, GPU memory が許せば範囲を広げる. SD U-Net は eval mode かつ凍結する. ただし student branch の input gradient は保持する. 通常の diffusion training は真の forward state  $v_t$  を入力する一方, 推論時は model 自身が生成した reverse state を使う. この差を検査するため, Phase 2 後半では  $t_*$  から  $t_H$  まで短く rollout した state 上でも handover/score loss を測る. まず rollout を detach した安定な学習を行い, 必要なら truncated backpropagation へ進む.

■Phase 3 : 任意の joint fine-tuning Phase 2 が baseline を上回った後に限り, ADJSCC decoder を小さい learning rate で同時更新する. encoder, VAE, SD U-Net は当面凍結する. 最初から end-to-end 更新すると, どの module が分布を直したか判別できず, SD prior へ過適合する危険がある.

## 7.3 推奨する batch 内 sampling

- SNR は  $\{-10, -5, 0, 5, 10, 20\}$  dB の離散一様, または  $[-10, 20]$  dB の連続一様を比較する.
- 低 SNR が難しすぎる場合は curriculum として  $20 \rightarrow 10 \rightarrow 0 \rightarrow -10$  dB へ範囲を広げる.
- timestep は一様 sample を baseline とし, handover 近傍を 50%, その他を 50% とする importance sampling を比較する.
- text condition はまず empty prompt に固定する.
- mixed precision 使用時も  $\bar{\alpha}_t$ , covariance, loss 集計は float32 以上で行う.

## 8 推論手順

### 8.1 共有 schedule・明示 bridge 方式

入力  $\hat{z}_0$ , channel SNR  $\gamma$ , handover 時刻  $t_H$  に対し, 推論は次の順序で行う.

1.  $\epsilon \sim \mathcal{N}(0, I)$  を sample し, 式 (4) で  $v_{t_\star}$  を初期化する.
2. Resfusion sampler で  $t_\star$  から  $t_H$  まで逆向きに更新する.
3. handover 点で  $\hat{R}_\theta$  を推定し, 式 (10) で  $z_{t_H}^H$  を作る.
4. Stable Diffusion sampler へ  $(z_{t_H}^H, t_H, c)$  を渡し, raw timestep  $t_H$  から 0 まで逆拡散する.
5. 得られた  $\hat{z}_0$  を scale 済み latent として VAE decoder へ渡す.

DDIM[3]/PLMS で step を間引く場合も, scheduler 配列の位置と raw timestep を区別する. 両 model へ渡す time embedding,  $\bar{\alpha}_t$ , handover index は同じ raw  $t$  を参照しなければならない. まず連続 DDPM step で数式の正しさを確認し, 次に共通の DDIM grid へ置き換えて速度を評価する.

### 8.2 疑似コード

```
z0          = vae_encode_scaled(image)          # teacher, training only
z_hat0      = adjsc_decode(channel(adjsc_encode(z0), snr))

# inference begins with received latent z_hat0
v = sqrt(alpha_bar[t_star]) * z_hat0 \
    + sqrt(1 - alpha_bar[t_star]) * randn_like(z_hat0)

for t in resfusion_grid(t_star, t_handover):
    eta_hat, r_hat = latent_resfusion(v, z_hat0, t, snr)
    v = resfusion_reverse_step(v, eta_hat, t)

eta_hat, r_hat = latent_resfusion(v, z_hat0, t_handover, snr)
z_handover = v - (1 - sqrt(alpha_bar[t_handover])) * r_hat

z_out = stable_diffusion_reverse(
    z_handover, start_raw_timestep=t_handover,
    condition=empty_prompt, guidance_scale=1.0
)
image_out = vae_decode_scaled(z_out)
```

## 9 評価計画

### 9.1 比較系

1. **ADJSCC only**:  $\hat{z}_0$  を直接 VAE decode する.
2. **SD only**: 通常 forward diffusion した  $\hat{z}_0$  から SD で復元する.
3. **full Resfusion**: handover せず Resfusion だけで  $t = 0$  まで復元する.
4. **direct handover**: bridge なしで  $v_{t_H}$  を SD へ渡す.
5. **bridge handover**: 式 (10) を用いる提案法.
6. **bridge + score**: さらに  $\mathcal{L}_{\text{score}}$  を用いる.
7. **bridge + score + stat**: 全補助 loss を用いる.

direct handover は式 (9) の影響を実証する重要な ablation である.

### 9.2 測定量

#### ■latent fidelity

$$\text{MSE}(\tilde{z}_0, z_0), \quad \text{NMSE} = \frac{\|\tilde{z}_0 - z_0\|_2^2}{\|z_0\|_2^2}.$$

global/channel mean, std, RMS, min/max,  $4 \times 4$  covariance, 共分散固有値,  $r_{\text{eff}}$  も既存 script と同じ定義で再測定する.

#### ■handover alignment

$$\mathbb{E} \|z_t^H - z_t^{\text{teacher}}\|_2^2, \quad e_{\text{cov}}, \quad \mathbb{E} \left\| \epsilon_{\psi}^{\text{SD}}(z_t^H, t, c) - \epsilon_{\psi}^{\text{SD}}(z_t^{\text{teacher}}, t, c) \right\|_2^2.$$

最後の量を「score gap」と呼び、本研究の仮説に最も直接対応する指標とする.

■image fidelity/perception PSNR, SSIM, LPIPS に加え、顔 dataset では identity similarity を測る. FID だけでは入力との対応を失った hallucination を見逃すため, paired metric と目視比較を必ず併用する.

■通信・計算量 各 SNR に対する性能, Resfusion step 数, SD step 数, 総 latency, peak VRAM, parameter 数を報告する.

### 9.3 主要 ablation

- $t_H \in \{500, 250, 100\}$
- shared schedule bridge と short-schedule re-noise bridge
- residual head あり/なし
- $\mathcal{L}_H$ ,  $\mathcal{L}_{\text{score}}$ ,  $\mathcal{L}_{\text{stat}}$  の有無
- SNR conditioning あり/なし
- fixed SNR 学習と mixed SNR 学習

- stochastic DDPM と deterministic DDIM
- empty prompt, guidance scale 1 と通常 CFG

## 10 成功条件と失敗の切り分け

### 10.1 段階ごとの成功条件

1. **bridge の代数検証**: 真の  $R$  なら handover MSE が浮動小数点誤差になる.
2. **分布補正**: 推定  $\hat{R}_\theta$  でも global/channel 統計と covariance error が ADJSCC baseline より改善する.
3. **U-Net 整合**: score gap が direct handover より有意に小さくなる.
4. **復元改善**: paired image metric と identity を保ったまま artifact が減る.
5. **SNR 頑健性**: 未知または中間 SNR でも性能が急落しない.

### 10.2 想定される失敗と診断

| 症状                            | 主な確認事項                                                                                                     |
|-------------------------------|------------------------------------------------------------------------------------------------------------|
| 統計は合うが identity が変わる          | moment loss が強すぎる, handover $t$ が高すぎる, CFG が強すぎる, sample alignment loss 不足.                                |
| latent MSE は下がるが artifact が残る | SD U-Net の score gap, text condition, sampler timestep のずれを確認する.                                           |
| score loss が学習されない            | student 側 U-Net forward を no-grad にしていないか, $z_t^H$ を detach していないか確認する.                                    |
| SNR ごとに性能が大きく変わる              | SNR embedding, channel noise variance, power normalization, train/test SNR 定義を確認する.                        |
| 出力が飽和する                       | latent $\rightarrow$ RGB 用 clamp や $[-1, 1]$ mapping を適用していないか確認する.                                        |
| handover 直後に品質が崩れる            | Resfusion と SD の schedule, raw timestep, $\bar{\alpha}_t$ , prediction parameterization (epsilon/v) を確認する. |

## 11 実装へ落とす際の変更点

公式実装を参照しつつ, 少なくとも以下を別 module として実装する.

- **latent distribution**: Resfusion/model/distributions.py の式を 4-channel latent  $\rightarrow$  適用する.
- **network**: Resfusion/model/denoising\_module/RDDM/RDDM\_model.py の input を 8 チャンネル, output を 8 チャンネルにし, SNR conditioning を加える.
- **training**: Resfusion/model/resfusion\_restore.py の RGB mapping, clamp, RGB PSNR を除き, bridge/score/stat losses を追加する.
- **scheduler**: 最初は Stable Diffusion model に登録済みの  $\alpha_t, \bar{\alpha}_t, \beta_t$  を唯一の source of truth と

して使う.

- **handover sampler**: raw timestep を明示して既存 DDIM/PLMS sampler へ途中 state を渡す adapter を作る.

checkpoint には Resfusion 重みだけでなく, VAE/ADJSCC/SD checkpoint hash, schedule 設定, VAE scale, SNR sampling 範囲,  $t_*$ ,  $t_H$ , prediction type を保存する.

## 12 研究上の注意

- Resfusion 原論文 [1] の実験は RGB restoration であり, 論文自身も latent-space extension を将来課題としている. 本メモの latent 版, two-head bridge, SD score distillation は新規設計であり, 原論文の主張として扱わない.
- forward diffusion 後に global statistics が Gaussian へ近づくことだけでは, U-Net の training distribution へ sample-conditionally 戻ったとは言えない. score gap と paired 復元性能で確認する.
- Stable Diffusion が FFHQ 専用 prior ではない場合, SD 側の domain gap も混在する. clean  $z_t$  を入力した oracle SD 復元を上限比較として必ず置く.
- Resfusion repository[5] は CC BY-NC-SA 4.0 を掲示しているため, code 再利用・配布時には license 条件を確認する.

## 13 最小実験の実行順

1. 真の  $R$  を使う oracle bridge を実装し, 式 (10) と SD 途中開始が正しいことを 100 画像で確認する.
2.  $t_H = 500, 250, 100$  の oracle score gap と復元品質を比較し, handover 候補を 1 つに絞る.
3. two-head Latent Resfusion を  $\mathcal{L}_{RN} + \mathcal{L}_R + \mathcal{L}_H$  で学習する.
4. direct handover と bridge handover を 1000 画像・全 SNR で比較する.
5.  $\mathcal{L}_{\text{score}}$  を追加し, score gap と artifact の因果を確認する.
6. その後にのみ stat loss, image loss, DDIM acceleration, ADJSCC decoder joint tuning を追加する.

この順序なら, 「Resfusion が効いた」という一括評価ではなく, 分布補正, U-Net 整合, 画像改善のどの段階で効果が生じたかを切り分けられる.

## 参考文献

- [1] Y. Liu et al., “Resfusion: Prior Residual Noise Embedded Denoising Diffusion Probabilistic Models for Image Restoration,” *NeurIPS 2024*. <https://arxiv.org/abs/2311.14900>
- [2] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” *NeurIPS 2020*. <https://arxiv.org/abs/2006.11239>
- [3] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” *ICLR 2021*. <https://arxiv.org/abs/2010.02502>

[//arxiv.org/abs/2010.02502](https://arxiv.org/abs/2010.02502)

- [4] R. Rombach et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” *CVPR 2022*. <https://arxiv.org/abs/2112.10752>
- [5] NKICSL, “Official implementation of Resfusion.” <https://github.com/nkicsl/Resfusion>